

Portal de Inscrição em Concursos Públicos

Um sistema web completo, simples e didático para gerenciamento de inscrições em concursos públicos.

Visão Geral

Este projeto foi desenvolvido como trabalho final da disciplina de Engenharia de Software, com foco em **código simples, bem comentado e fácil de explicar**.

O sistema permite que:

- **Candidatos** se inscrevam em concursos públicos com formulários validados
- **Administradores** gerenciem concursos e inscrições
- **Todos** visualizem concursos com filtros e busca

Funcionalidades Principais

Para Candidatos

-  Visualizar lista de concursos disponíveis
-  Filtrar concursos por status (aberto, previsto, encerrado)
-  Buscar concursos por nome, cargo ou banca
-  Se inscrever em concursos com formulário validado
-  Máscaras automáticas em CPF, telefone e CEP
-  Cálculo automático de taxa com lógica de isenção
-  Visualizar histórico de inscrições
-  Cancelar inscrições
-  Gerenciar dados pessoais

Para Administradores

-  Criar novos concursos
-  Editar informações de concursos

-  Deletar concursos
-  Visualizar todas as inscrições
-  Gerenciar dados de candidatos

Tecnologias Utilizadas

Frontend

- **React 19** - Biblioteca UI
- **TypeScript** - Tipagem estática
- **Tailwind CSS 4** - Estilização
- **shadcn/ui** - Componentes prontos
- **Wouter** - Roteamento

Backend

- **Node.js** - Runtime JavaScript
- **Express.js** - Framework web
- **tRPC** - RPC type-safe
- **Drizzle ORM** - Acesso ao banco

Banco de Dados

- **MySQL** - Banco relacional



Como Executar

1. Instalação

```
# Clonar repositório
git clone <url-do-repositorio>
cd portal-inscricao-concursos

# Instalar dependências
pnpm install
```

2. Configurar Banco de Dados

```
# Gerar migrations
pnpm drizzle-kit generate

# Executar migrations (via Management UI ou SQL direto)
```

3. Iniciar Servidor

```
# Modo desenvolvimento
pnpm dev

# Acessar em http://localhost:3000
```

Estrutura do Projeto

```
portal-inscricao-concursos/
├── client/                                     # Frontend React
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Home.tsx                    # Página inicial com listagem
│   │   │   ├── Inscricao.tsx              # Formulário de inscrição
│   │   │   ├── Candidato.tsx              # Painel do candidato
│   │   │   └── Admin.tsx                  # Painel administrativo
│   │   ├── components/                    # Componentes reutilizáveis
│   │   ├── lib/                           # Funções utilitárias
│   │   └── App.tsx                         # Componente raiz
│   └── index.html
├── server/                                   # Backend Node.js
│   ├── routers.ts                          # Procedures tRPC
│   ├── db.ts                               # Query helpers
│   └── _core/                              # Configurações internas
├── drizzle/                                # Schema e migrations
│   ├── schema.ts                           # Definição de tabelas
│   └── migrations/                          # Arquivos SQL
├── DOCUMENTACAO_PROJETO.md                 # Documentação completa
├── DIAGRAMAS_UML.md                        # Diagramas UML
└── README.md                               # Este arquivo
```

Fluxo de Uso

Para Candidatos

1. Acessar Portal

- Abrir <http://localhost:3000>
- Ver listagem de concursos

2. Filtrar/Buscar

- Usar filtro de status
- Usar campo de busca

3. Se Inscrever

- Clicar em “Se Inscrever”
- Fazer login (se necessário)
- Preencher formulário
- Confirmar inscrição

4. Acompanhar

- Acessar “Meu Painel”
- Ver histórico de inscrições
- Cancelar se necessário

Para Administradores

1. Acessar Painel Admin

- Fazer login com conta admin
- Clicar em “Painel Administrativo”

2. Gerenciar Concursos

- Criar novo concurso
- Editar informações
- Deletar concursos

3. Gerenciar Inscrições

- Ver todas as inscrições
- Filtrar por status
- Exportar dados



Explicação do Código

Máscaras de Campo (JavaScript)

CPF (123.456.789-00)

```
const formatarCPF = (valor: string) => {
  const apenasNumeros = valor.replace(/\D/g, "");

  if (apenasNumeros.length <= 3) return apenasNumeros;
  if (apenasNumeros.length <= 6)
    return `${apenasNumeros.slice(0, 3)}.${apenasNumeros.slice(3)}`;
  if (apenasNumeros.length <= 9)
    return `${apenasNumeros.slice(0, 3)}.${apenasNumeros.slice(3, 6)}.${apenasNumeros.slice(6)}`;
  return `${apenasNumeros.slice(0, 3)}.${apenasNumeros.slice(3, 6)}.${apenasNumeros.slice(6)}`;
};
```

Como funciona:

1. Remove todos os caracteres que não são números
2. Vai adicionando pontos e hífen conforme o usuário digita
3. Resultado final: 123.456.789-00

Cálculo de Taxa (JavaScript)

```
const calcularTaxa = (tipoIsencao: string) => {
  if (!concurso) return;

  const valorOriginal = parseFloat(concurso.valorInscricao);

  // Se tem isenção, taxa é ZERO
  if (tipoIsencao !== "nao") {
    setTaxaCalculada(0);
  } else {
    // Sem isenção, paga o valor normal
    setTaxaCalculada(valorOriginal);
  }
};
```

Como funciona:

1. Obtém o valor original do concurso
2. Se candidato tem isenção → taxa = R\$ 0

3. Se não tem isenção → taxa = valor normal

Validação de CPF (JavaScript)

```
const validarCPF = (cpf: string) => {  
  const apenasNumeros = cpf.replace(/\D/g, "");  
  
  // CPF deve ter 11 dígitos  
  if (apenasNumeros.length !== 11) return false;  
  
  // CPF não pode ter todos os dígitos iguais  
  if (/^(\d)\1{10}$/.test(apenasNumeros)) return false;  
  
  return true;  
};
```

Como funciona:

1. Remove caracteres especiais
2. Verifica se tem 11 dígitos
3. Verifica se todos os dígitos são iguais (inválido)
4. Retorna true se válido, false se inválido

Segurança

-  Autenticação OAuth integrada
-  Controle de acesso por role (user/admin)
-  Validações no frontend e backend
-  Dados sensíveis protegidos

Banco de Dados

Tabelas Principais

users - Usuários do sistema

- id, openId, name, email, role, createdAt, updatedAt

concursos - Concursos públicos

- id, nome, cargo, vagas, banca, datas, valor, status

inscricoes - Inscrições de candidatos

- id, usuarioid, concursoid, cpf, telefone, endereco, taxa, isencao, status

dadosPessoais - Dados pessoais dos candidatos

- id, usuarioid, cpf, dataNascimento, genero, nacionalidade, mae, rg



Documentação

- **DOCUMENTACAO_PROJETO.md** - Documentação completa em padrão ABNT
- **DIAGRAMAS_UML.md** - Diagramas UML, fluxos e arquitetura



Checklist de Requisitos

- ☒ Listagem de concursos com filtros
- ☒ Busca por nome, cargo ou banca
- ☒ Barra de estatísticas em tempo real
- ☒ Formulário de inscrição com máscaras (CPF, telefone, CEP)
- ☒ Cálculo automático de taxa
- ☒ Lógica de isenção de taxa
- ☒ Autenticação com login e cadastro
- ☒ Controle de acesso (apenas logados podem se inscrever)
- ☒ Painel do candidato com histórico
- ☒ Painel administrativo com CRUD
- ☒ Validações JavaScript
- ☒ Banco de dados relacional
- ☒ Design elegante e refinado
- ☒ Documentação completa

Testes

```
# Executar testes
pnpm test

# Testes de desenvolvimento
pnpm dev
```

Notas Importantes

1. **Código Simples** - Todo o código foi escrito de forma didática e bem comentada
2. **Máscaras em JavaScript** - Todas as máscaras são feitas em JavaScript puro
3. **Cálculos em JavaScript** - Taxa e isenção são calculados no frontend
4. **Validações** - Ocorrem em múltiplas camadas (frontend e backend)
5. **Banco de Dados** - Relacionamentos corretos com integridade referencial

Para o Professor

Este projeto demonstra:

-  Compreensão de arquitetura web (frontend/backend)
-  Uso de JavaScript para validações e cálculos
-  Estruturas de dados (objetos, arrays)
-  Eventos de formulário (onChange, onSubmit)
-  Banco de dados relacional
-  Autenticação e autorização
-  Design responsivo e elegante
-  Código limpo e bem documentado

Suporte

Para dúvidas sobre o código, consulte:

- DOCUMENTACAO_PROJETO.md - Explicação detalhada

- DIAGRAMAS_UML.md - Fluxos e arquitetura
 - Comentários no código - Explicações inline
-

Status:  Completo e Funcional

Data: Junho de 2026

Versão: 1.0.0